

中国科技大学Verilog实验课程

中国科技大学Verilog实验课程

Chapter I 基础概念

归约运算符与按位运算符

移位运算符

拼接运算符

连续赋值: **assign**

过程赋值: **always**

阻塞赋值和非阻塞赋值

Verilog的基本结构

模块例化

参数传递

Chapter II Verilog 的描述层次

从结构化层面描述电路

数据流描述方式

行为级描述方式

时钟信号和复位信号

尽量使用同步复位的原因

搭建**Logisim**教程

使用**VScode**编辑

几点注意事项

Chapter III 测试与仿真

再遇**Verilog**

关于阻塞赋值

避免锁存器: 逻辑缺陷, 自引用与默认赋值

编写测试文件

Testbench

循环

测试文件中的系统任务

计时器测试文件实例

最大值测试文件实例

Chapter IV 上板运行

消抖:

边沿检测:

计数器分频

IP分频

有限状态机

可复位时钟
移位寄存器
交通信号灯
投币机
超前进位加法器

Chapter I 基础概念

关系运算符的运算结果固定为 0 (False) 或 1 (True) , 是一个 1bit 的值。如果某一个操作数中有一位为 x 或 z, 则前六种关系运算的结果为 x。全等比较则对为 x 或 z 的位也进行比较, 只有两个操作数完全一致时, 其结果才是 1, 否则结果是 0。例如:

```
(4'b1010 == 4'b101x) = x
(4'b101z == 4'b1010) = x
(4'b1010 === 4'b101x) = 0
(4'b101z === 4'b1010) = 0
(4'b101z === 4'b101z) = 1
```

归约运算符与按位运算符

按位运算符对 2 个操作数的每 bit 数据进行按位操作。如果 2 个操作数位宽不相等, 则用 0 向左扩展补充较短的操作数。按位非是单目运算符, 它对操作数的每 bit 数据进行取反操作。例如:

```
A = 4'b0101;
B = 4'b1001;

~A      // 4'b1010
A & B   // 4'b0001
A | B   // 4'b1101
A ^ B   // 4'b1100
```

1. `&` (归约与)
2. `&&` (归约非与)
3. `|` (归约或)
4. `~|` (归约非或)
5. `^` (归约异或)
6. `~^` (归约同或)

归约运算符与按位运算符的符号相同，但归约运算符都是单目运算符。它对多位操作数逐位进行操作，最终产生一个 1bit 结果。例如：

```
A = 4'b1010;
&A          // 结果为 1 & 0 & 1 & 0 = 1'b0, 可用来判断变量 A 是否全 1
~|A         // 结果为 ~(1 | 0 | 1 | 0) = 1'b0, 可用来判断变量 A 是否为全 0
^A          // 结果为 1 ^ 0 ^ 1 ^ 0 = 1'b0
```

移位运算符

1. `<<` (逻辑左移)
2. `>>` (逻辑右移)
3. `<<<` (算数左移)
4. `>>>` (算数右移)

在算术左移和逻辑左移时，右边低位会补 0。在逻辑右移时，左边高位会补 0；而算术右移时，左边高位会补充符号位，以保证数据缩小后值的正确性。例如：

```
A = 8'b1100_1101;
A >> 1        // 逻辑右移，补 0。8'b0110_0110
A >>> 2       // 算数右移，补符号位。8'b1111_0011
A << 2        // 逻辑左移，补 0。8'b0011_0100
```

拼接运算符

```
a[7:4]={a[0], a[1], a[2], a[3]};
```

表示将 a 信号的低 4 位颠倒并赋值给高 4 位,

```
b[31:0]={24{a[7]}, a[7:0]};
```

连续赋值：assign

```
wire Cout, A, B;  
assign Cout = A & B;
```



这里有一些语法细节：

- LHS 必须是一个 wire 型变量，而不能是 reg 类型的变量；
- RHS 的类型没有要求；
- 只要 RHS 表达式的操作数有事件发生（值的变化）时，RHS 就会立刻重新计算，同时赋值给 LHS。这体现了 Verilog 语言的硬件特征：assign 语句实际上是构建了一段门电路，会长期存在于数字系统之中。

Verilog 还提供了另一种对 wire 型赋值的简单方法，即在 wire 型变量声明的时候同时对其赋值。例如，下面的赋值方式和上面赋值例子的效果是一致的。

```
wire A, B;  
wire Cout = A & B;
```



过程赋值：always

过程赋值语句用于对 reg 型变量进行赋值，由 2 种关键字引导，分别是 initial 与 always。这两种语句不可嵌套使用，彼此间并行执行（执行的顺序与其在模块中的前后顺序没有关系）。如果 initial 或 always 语句内包含多个语句，则需要搭配关键字 begin 和 end 组成一个块语句。

每个 initial 语句或 always 语句都会产生一个独立的控制流，执行时间都是从 0 时刻开始。二者的区别在于 initial 仅在 0 时刻开始执行一次内部的语句，而 always 语句块从 0 时刻开始执行，当执行完最后一条语句后，便再次执行语句块中的第一条语句，如此循环反复。

实践证明：没有任何条件限制的 `always` 并不是那么好用。因此，`always` 引入了**敏感变量**的概念。我们常常以下面的格式使用 `always` 语句：

```
always @(敏感变量列表) 过程语句
```

敏感变量就是触发 `always` 块内部语句的条件。加入敏感变量后，`always` 语句仅在列表中的变量发生变化时才执行内部的过程语句。

```
// 每当 a 或 b 的值发生变化时就执行内部的语句
always @(a or b) begin
    [过程语句]
end
```

阻塞赋值和非阻塞赋值

⚠ 注意：非阻塞赋值

我们考虑下面这段代码：

```
1  reg [4:0] a;
2  always @(posedge clk) begin
3      a <= a + 1;
4      a <= a + 2;
5  end
```

它会如何执行呢？`always` 语句在 `clk` 信号的上升沿触发，但其内部是对同一个变量的两次非阻塞赋值。实际上，对于非阻塞赋值来说，只有最后一次的赋值是有效的，因此每次 `clk` 上升沿时变量 `a` 的值都会增加 2，而不是 1。

作为对比，我们来看下面这段代码：

```
1  reg [4:0] a;
2  always @(posedge clk) begin
3      a = a + 1;          // <- 不要这么写！在时序模块里面只用非阻塞赋值！
4      a = a + 2;          // <- 这里仅作为一个例子，大家不要学习这样的写法！
5  end
```

阻塞赋值是可以叠加的。因此每次 `clk` 上升沿时 `a` 的值增加 3。

注意！

养成使用 `begin/end` 关键字的习惯

`input`、`inout` 类型的端口不能声明为 `reg` 数据类型，因为 `reg` 类型常用于保存数值，而输入端口只反映与其相连的外部信号的变化，不应保存这些信号的值。`output` 类型的端口则可以声明为 `wire` 或 `reg` 数据类型。

在 Verilog 中，`wire` 型为默认数据类型，因此当端口为 `wire` 型时，不用再次声明端口类型为 `wire`；但是当端口为 `reg` 型时，对应的 `reg` 声明不可省略。

Verilog的基本结构

```
module FA (  
    input      a, b, cin,  
    output     cout,  
    output reg  s  
);
```

```
module 模块名 (  
    // 端口定义之间用英文逗号 , 分隔开  
    输入端口定义,           // 输入端口只能是 wire 类型  
    输出端口定义           // 输出端口可以根据需要定义为 wire 或 reg 类型  
);                          // 不要忘记这里的分号  
  
    内部信号定义语句       // 内部信号可以根据需要定义为 wire 或 reg 类型  
    模块实例化语句         // 将其他模块接入电路  
    assign 数据流赋值语句  
    always 过程赋值语句  
endmodule
```

模块例化

首先定义一个模块

```
module FA (  
    input [7:0] a, b,  
    input      cin,  
    output reg [7:0] s,  
    output     cout  
);
```

在顶层模块中，我们定义如下变量：

```
wire [7:0] num1, num2, sum;  
wire cin, cout;
```

- 基于位置的端口关联（不便）
- 基于名字的端口关联（常用）

```
FA fa (.a(num1), .b(num2), .s(sum), .cin(cin), .cout(cout));
```



这种方法将需要例化的模块端口与外部信号按照其名字进行连接，端口顺序可以与引用 module 的声明端口顺序不一致，只要保证端口名字与外部信号匹配即可。如果某些输出端口并不需要在外部连接，例化时可以悬空不连接，甚至直接删除。一般来说，input 端口在例化时不能删除，否则编译报错，output 端口在例化时可以删除。

Tips: 端口的连接规则

在名字例化方式中，我们需要额外介绍端口的连接规则。

- **输入端口**

模块例化时，input 端口可以连接 wire 或 reg 型变量；模块声明时，input 端口必须是 wire 型变量。

- **输出端口**

模块例化时，output 端口必须连接 wire 型变量；模块声明时，output 端口可以是 wire 或 reg 型变量。

- **输入输出端口**

模块例化和模块声明时，inout 端口都必须连接 wire 型变量。

参数传递

在编写子模块时并不预先指定位宽，而是在例化的时候根据需要确定位宽。此时我们可以使用 Verilog 的带参数例化功能：模块声明时使用 `parameter` 关键字指定参数，例化时将新的参数值写入模块例化语句，以此来改写子模块的参数值。

```
module MUX2
# (
    parameter          WIDTH = 8
)(
    input [WIDTH-1: 0]  num1, num2,
    input               sel,
    output reg [WIDTH-1: 0] ans
);
always @(*) begin
    if (sel)
        ans = num1;
    else
        ans = num2;
end
endmodule
```

例化：

```
wire [3:0] num1, num2, ans;
wire sel;

MUX2 #(4) mux (
    .num1(num1),
    .num2(num2),
    .sel(sel),
    .ans(ans)
);
```

Chapter II Verilog 的描述层次

- 结构化描述方式：调用其他已经定义过的低层次模块对整个电路的功能进行描述，或者直接调用 Verilog 内部预先定义的基本门级元件描述电路的结构进行描述。
- 数据流描述方式：使用连续赋值语句 **assign** 对电路的逻辑功能进行描述。该方式特别适合于对组合逻辑电路建模。
- 行为级描述方式：使用过程块语句结构 **always** 和比较抽象的高级程序语句对电路的逻辑功能进行描述。

从结构化层面描述电路

Verilog 常用的内置逻辑门包括：

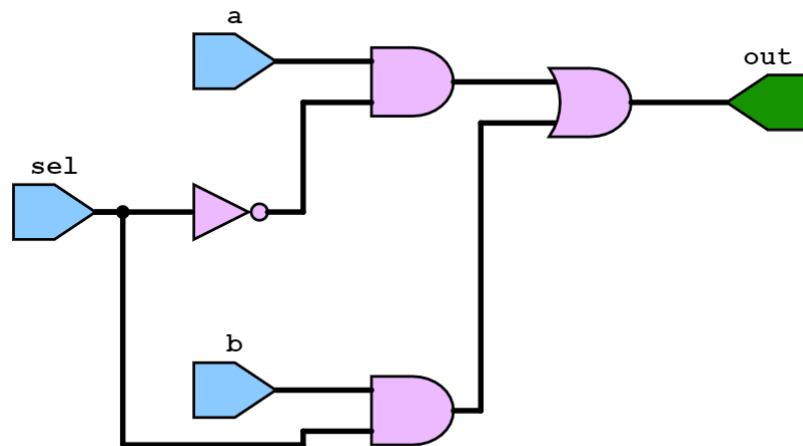
- and (与门)
- nand (与非门)
- or (或门)
- nor (或非门)
- xor (异或门)
- xnor (同或门)

我们可以通过类似模块例化的方式使用这些逻辑门，进而实现一些简单的逻辑功能。

Tips：逻辑门的例化

由于不知道输入信号的数目，门级单元无法采用基于名字的端口关联方式，只能基于位置进行端口关联。一般来说，门级单元的第一个端口是输出，后面其余的端口是输入。在例化调用的时候，我们也可以不指定门级单元实例的名字，从而为代码编写提供了方便（毕竟逻辑门的数目可能很多）。当输入端口超过 2 个时，只需要将输入信号在端口列表中继续排列即可，Verilog 可以自动识别输入和输出信号。

例如。使用如下语句对电路进行结构级描述：



```
1  module MUX2(  
2      input    a, b,  
3      input    sel,  
4      output   out  
5  );  
6  
7  wire and1, and2, sel_not;  
8  
9  and (and1, a, sel_not);  
10 and (and2, b, sel);  
11 not (sel_not, sel);  
12 or (out, and1, and2);  
13  
14 endmodule
```

数据流描述方式

```
module MUX2(  
    input    a, b,  
    input    sel,  
    output   out  
);  
  
assign out = (a & ~sel) | (b & sel);  
endmodule
```

可以看到，得出逻辑表达式后使用 `assign` 语句进行赋值

行为级描述方式

很多时候，我们难以得到模块的电路结构，或者得到的结构十分繁琐，这时我们就可以使用行为级描述，以类似于高级语言的抽象层次进行硬件结构开发。这一层面的描述过程更看重功能需求与算法实现，也是对于我们最为友好的描述方式。

```
module MUX2(  
    input    a, b,  
    input    sel,  
    output reg out  
);  
always @(*) begin  
    if (!sel)  
        out = a;  
    else  
        out = b;  
end  
endmodule
```

时钟信号和复位信号

```
parameter T_high = 10;  
parameter T_low = 5;  
reg clk;  
always begin  
    clk = 1;  
    # T_high;  
    clk = 0;  
    # T_low;  
end
```

下面的代码是错误的！

```

module Init4wire (
    input    a, b, c,
    output   out
);
wire temp = 0;
assign temp = a | b;
assign out = temp | c;
endmodule

```

第一句 `temp=0` 实际上等价于 `assign temp = 0` 故只有第一句起作用！第二句是无用的！

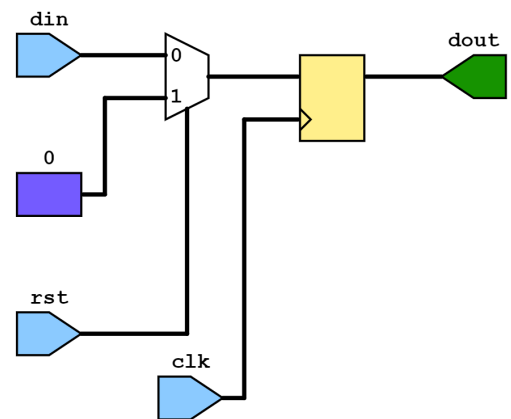
为确保实际环境下数字系统在上电后有一个明确、稳定的初始状态，且系统在运行紊乱时可以恢复到正常的初始状态，我们会在模块设计中添加复位模块。复位电路保证了系统工作的可控性，在一定程度上其重要性不亚于时钟信号。

从时序上来看，复位电路可分为同步复位（**Synchronous Reset**）和异步复位（**Asynchronous Reset**）两种。

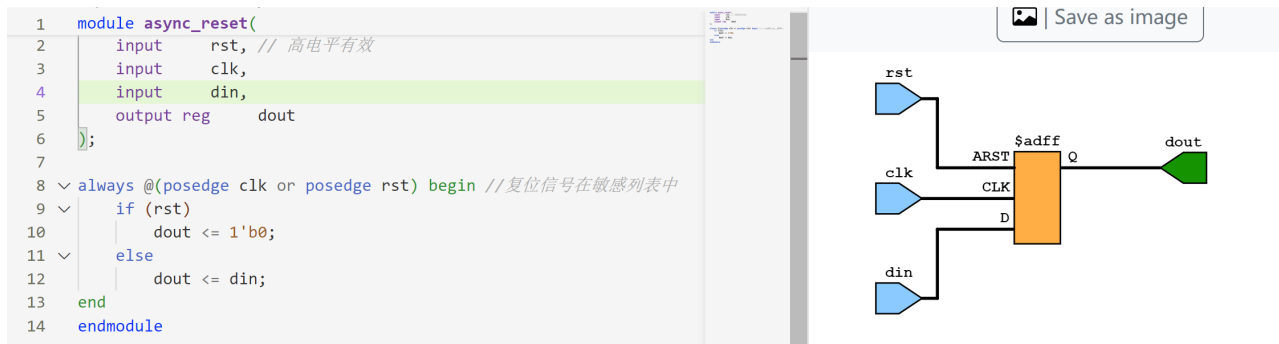
```

1  module sync_reset(
2      input    rst, // 高电平有效
3      input    clk,
4      input    din,
5      output reg dout
6  );
7  always @(posedge clk) begin // 复位信号不在敏感列表中
8      if (rst)
9          dout <= 1'b0;
10     else
11         dout <= din;
12     end
13 endmodule

```



同步复位保证了信号是与时钟同步变化的，有利于保证时序的稳定性，因此被广泛使用。但大多数时序逻辑单元并没有同步复位端，使用同步复位描述得到的电路往往会消耗更多的逻辑资源。此外，复位信号的宽度必须大于一个时钟周期，否则便有可能发生遗漏。



异步复位会导致复位信号与时钟信号之间没有明确的时序关系，并且复位信号容易受到外部因素的干扰，产生意想不到的复位操作。

尽量使用同步复位的原因

```

module Reset_test(
    input          clk,
    input          rst, // 复位信号，高电平有效
    input          en,  // 写入控制信号
    input [15:0]   din,
    output reg [15:0] dout1,
    output reg [15:0] dout2
);

always @(posedge clk or posedge rst) begin
    if (en)
        dout1 <= din;
    else if (rst)
        dout1 <= 0;
end

always @(posedge clk or posedge rst) begin
    if (rst)
        dout2 <= 0;
    else if (en)
        dout2 <= din;
end
endmodule

```

右边两个黄色的矩形是自动生成的寄存器（也就是时序逻辑单元），从名字可以看出上面的对应 `dout1`，下面的对应 `dout2`。注意观察左侧 `rst` 信号的连接情况，我们发现 `dout1` 并没有与 `rst` 信号相连！此外，`dout2` 寄存器也比 `dout1` 寄存器多了一个 `CLR` 端口。这表明 `dout1` 并没有将 `rst` 视作异步复位信号。

为什么会这样呢？实际上这是 `if-else` 语句的优先级导致的。在 `dout1` 对应的描述里，`rst` 信号比 `en` 信号判断的优先级低。也就是说，如果 `en` 信号为高电平，那么在 `rst` 的上升沿到来时，`dout1` 信号先会进入 `if (en)` 的判断，结果为真，从而执行 `dout1 <= din;` 而不是复位操作 `dout1 <= 0;`，这显然不符合异步复位的要求。所以在这里编译器将 `rst` 视作了同步复位信号。

搭建Logisim教程

[Logisim 使用教程 - Digital Lab 2024 \(ustc.edu.cn\)](#)

使用VScode编辑

[Verilog 编程 - Digital Lab 2024 \(ustc.edu.cn\)](#)

几点注意事项

- 在进行模块例化时，无论端口数目多少，请一定使用基于端口名称的关联
- 阻塞赋值 `=` 只用在 `assign` 和 `always @(*)` 的组合逻辑中，非阻塞赋值 `<=` 只用在 `always @(posedge clk)` 的时序逻辑中。请不要在错误的场景下使用，甚至二者混用。
- 不要使用 `for` 循环

```
// 括号中的代码需要缩进
cache_memory way0 (
    .addra (w_index), // Tab 缩进
    .clka (clk),
    .dina (mem_din),
    .ena (mem_en[0]),
    .wea (mem_we),
    .addrb (r_index),
    .doutb (mem_dout0)
); // 分号不要忘记

// begin...end 中的代码需要缩进
always @(*) begin
    case (wrt_data_sel) // Tab 缩进
        1'b0: mem_din = w_data_AXI;
        1'b1: begin
```

```

    case(wrt_type) // Tab 缩进
        BYTE: mem_din = {64{w_data_CPU[7:0]}}; // Tab 缩进
        HALF: mem_din = {32{w_data_CPU[15:0]}};
        WORD: mem_din = {16{w_data_CPU}};
        default: mem_din = 0;
    endcase // 与 17 行的 case 对齐
end // 与 1'b1 对齐, 因为这个 end 对应 16 行的 begin
endcase // 与 14 行的 case 对齐
end // 与 always 对齐

```

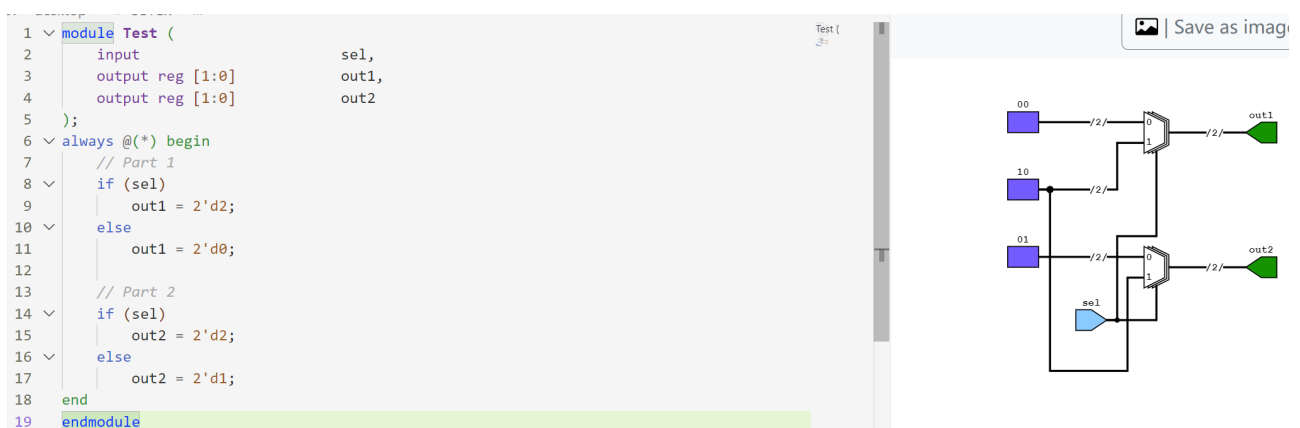
Chapter III 测试与仿真

再遇Verilog

关于阻塞赋值

在同一个 `always` 块里的阻塞赋值语句也可以是并行执行的，只要其内部的信号不会产生冲突。

作为一门硬件描述语言，我们一直在强调：**Verilog** 中的每一条语句都对应着一种实际的硬件结构，例如 `if` 语句对应的是选择器。



如上，虽然是 阻塞赋值 但是当 `sel` 信号发生变化时，信号 `out1` 和 `out2` 是同步变化的！

如果两个 `if` 的赋值对象没有冲突，那么两个 `if` 描述的多选器是并行的，否则是串行的。

避免锁存器：逻辑缺陷，自引用与默认赋值

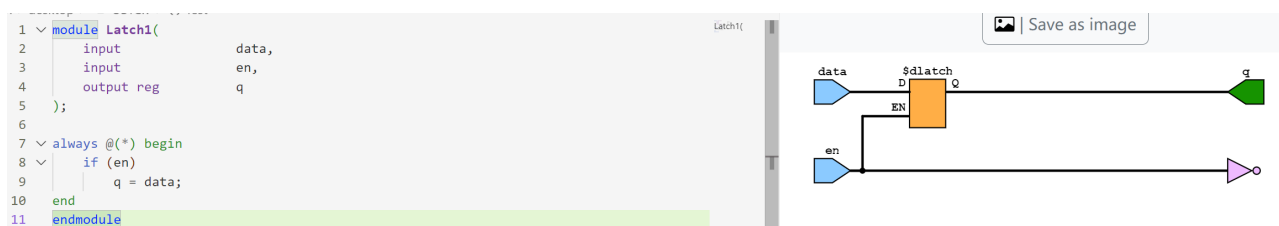
在 Verilog 中，一个变量如果声明为寄存器类型（reg），它既可以被综合成组合逻辑的导线，也可能被综合成时序逻辑中的寄存器或锁存器。在使用 `always @(*)` 语句时，我们会希望变量被综合成导线，但是有时候由于代码书写问题，它会被综合成我们不期望的锁存器结构，进而对电路带来危害。主要有：

- 电路的输出状态可能发生多次变化，增加了下一级电路的不确定性；
- 在大部分 FPGA 的设计里，锁存器结构会消耗更多的电路资源；
- 锁存器导致电路不能按照我们预期的方式工作，在调试时带来额外的问题。

因此，我们在代码书写时需要格外注意，应当避免出现锁存器。一个简单且好记的原则是：组合逻辑中不应出现记忆电路，即电路不能保存自身的状态。违反了这一原则的组合逻辑电路往往就会产生锁存器。

产生 latch 的原因有以下几种：

在组合逻辑中 `if else` 的缺陷：



当 `en` 信号为 1 时，`q` 会被赋值为 `data` 的值；当 `en` 信号不变时，由于 `always` 语句里的 `if` 缺少对应的 `else` 分支，因此编译器默认 `else` 的分支下寄存器 `q` 的值保持不变。此时电路应当具有存储数据的功能，所以变量 `q` 会被综合成锁存器结构。

- 特别注意！在时序逻辑中，不完整的 `if-else` 结构不会产生锁存器

另外，漏掉信号的某些情况也可能导致错误：

```
// 补全 if-else 分支结构  
always @(*) begin  
    if (en) begin  
        q1 = data1;  
        q2 = 1'b0;  
    end  
    else begin  
        q1 = 1'b0;  
        q2 = data2;  
    end  
end
```

```

        end
    end

    // 为 q1、q2 赋初值
    always @(*) begin
        q1 = 1'b0;
        q2 = 1'b0;
        if (en)
            q1 = data1;
        else
            q2 = data2;
    end
end

```

避免因case产生的错误:

```

// 使用 default 补充逻辑
always @(*) begin
    case(sel)
        2'b00:    q = data1;
        2'b01:    q = data2;
        default:  q = 1'b0;
    endcase
end

// 枚举补充逻辑
always @(*) begin
    case(sel)
        2'b00:    q = data1;
        2'b01:    q = data2;
        2'b10, 2'b11:
            q = 1'b0;
    endcase
end

// 使用默认赋值
always @(*) begin
    q = 1'b0;
    case(sel)
        2'b00:    q = data1;
        2'b01:    q = data2;
    endcase
end

```

end

在 **always** 语句块的一开始就进行默认赋值。这样可以避免潜在的逻辑不完整风险。

信号不要给信号自己赋值，且不要用赋值信号本身参与判断条件逻辑。否则也会产生 latch

编写测试文件

Testbench

由不可综合的 Verilog 代码组成，这些代码用于生成待测模块的输入，并验证待测模块的输出是否正确（是否符合预期）

- 激励（**Stimulus Block**）是专门为待测模块生成的输入。我们需要尽可能产生全面的测试场景，包括合法的和不合法的。
- 输出校验（**Output Checker**）用于检查被测模块的输出是否符合预期。
- 被测模块（**Design Under Test, DUT**。也称 **Unit Under Test, UUT**）是我们编写的 Verilog 模块，Testbench 的主要目的就是对其进行验证，以确保在特定输入下其输出均与预期一致。

编写 Testbench 的第一步是创建一个 Verilog 模块作为测试的顶层模块。与正常设计时的 Verilog module 不同，用于测试的模块应当没有输入和输出，这是因为 Testbench 模块应当是完全独立的，不受外部信号的干扰。

接下来，我们需要例化待测模块，将信号连接到待测模块以允许激励代码运行。这些信号包括时钟信号和复位信号，以及传入 Testbench 的测试数据。

```
module Module_tb ();

    // 定义并产生激励信号
    // .....

    Test_module #(
        // 参数接口
    ) test (
        // 待测模块端口
    );
endmodule
```

使用延时可能屏蔽掉信号的变化：

原因是：`assign #2 z = ~a;`可以拆成两部分：首先是立即执行的`assign temp = ~a;`，其次是延时两个时间单位执行的`assign z = temp;`。在这两个时间单位内，对变量 `a` 的改变将直接反应在变量 `temp` 上（还记得 `assign` 语句的特点吗？），而变量 `z` 并不会察觉到这一变化，也就不会做出相应的改变。

我们可以用如下的 `always` 语句进行替代：

```
1 always @(a) begin
2     temp = ~a;
3     #2;
4     z = temp;
5 end
```

这也体现出连续赋值与组合逻辑电路没有记忆性的特点。

除了直接控制延迟，Verilog 还支持基于事件的时间控制，这就是 `@` 符号的作用。Verilog 有以下三种常用的事件控制方式：

1. `@ + 信号名`，表示当信号发生逻辑变化时执行后面的内容。例如 `@ (in) out = in;`
2. `@ + posedge + 信号名`，表示当信号从低电平变化到高电平时执行后面的内容。例如 `@ (posedge in) out = in;`
3. `@ + negedge + 信号名`，表示当信号从高电平变化到低电平时执行后面的内容。例如 `@ (negedge in) out = in;`

这也印证了 `always` 语句的敏感变量语法。

与 `always` 块不同，在 `initial` 块中编写的 Verilog 代码几乎都是不可综合的，因此基本上只被用于仿真与初始化信号。

循环

除了 Lab1 中介绍的赋值语句和分支语句，Verilog 中也有循环语句。它们分别是 `while`、`for`、`repeat` 和 `forever` 循环。循环语句只能在 `always` 或 `initial` 块中使用，其内部可以包含延迟表达式。

```
integer i;
reg [3:0] counter;
initial begin
    counter = 'b0;
    for (i = 0; i <= 10; i = i + 1) begin
        #10;
        counter = counter + 1'b1;
    end
end
```

在 Verilog 语言里，`i = i + 1` 不能像 C 语言那样写成 `i++` 的形式，`i = i - 1` 也不能写成 `i--` 的形式。

```
reg [3:0] counter;
initial begin
    counter = 'b0;
    repeat (11) begin
        #10;
        counter = counter + 1'b1;
    end
end
```

测试文件中的系统任务

`$display` 允许我们在控制台上输出一条消息。该函数的使用方式与 C 语言中的 `printf` 函数非常类似，这意味着我们可以轻松地在 Testbench 中创建文本语句，并使用它们来显示有关仿真状态的信息。

此外，我们还可以在字符串中使用特殊字符 `%` 来规范化显示信号数值。我们需要使用一个格式字母来决定以何种格式显示变量数值，在格式代码前面加上一个数字来确定要显示的位数。最常用的格式是 `b`（二进制）、`d`（十进制）和 `h`（十六进制）。

<code>%b</code> 或 <code>%B</code>	显示为二进制
<code>%d</code> 或 <code>%D</code>	显示为十进制
<code>%h</code> 或 <code>%H</code>	显示为十六进制
<code>%o</code> 或 <code>%O</code>	显示为八进制格式
<code>%c</code> 或 <code>%C</code>	显示为 ASCII 字符
<code>%m</code> 或 <code>%M</code>	显示模块的层级名称
<code>%s</code> 或 <code>%S</code>	显示为字符串
<code>%t</code> 或 <code>%T</code>	显示为时间
在格式字母前加 0	根据实际位宽显示，不输出多余空格

`$display` 函数支持的格式

```

reg [4:0] x;
initial begin
    x = 0;
    repeat (10) begin
        // 分别用 2 进制、16 进制和 10 进制来打印 x 的值
        $display("x(bin) = %b, x(hex) = %h, x(decimal) = %d\n", x,
x, x);
        #10;
        x = x + 2;
    end
end
end

```

`$monitor` 函数与 `$display` 函数非常相似，但它一般被用来监视 Testbench 中的特定信号。这些信号中的任何一个改变状态，都会在终端打印一条消息。

```

reg [4:0] a, b;
initial begin
    a = 0;
    b = 20;
    repeat (10) begin
        #10;
        a = a + 2;
        b = b - 2;
    end
end

initial begin
    $monitor("now a = %d, b = %d\n", a, b);
end

```

使用 `$time` 获取系统仿真时间：

```

reg [4:0] a, b;
initial begin
    a = 0;
    b = 20;
    repeat (10) begin
        #10;
        a = a + 2;
        b = b - 2;
    end
end

```

```

        end
    end

    initial begin
        $monitor("Time %0t: a = %d, b = %d\n", $time, a, b);
    end
end

```

计时器测试文件实例

```

module Counter_tb();
    reg clk, rst;
    wire out;

    initial begin
        clk = 0;
        rst = 1;
        #50;
        rst = 0;
    end
    always #10 clk = ~clk;

    Counter #(8) my_counter (
        .clk(clk),
        .rst(rst),
        .out(out)
    );
endmodule

```

最大值测试文件实例

```

module MAX2_tb();
    parameter TEST_NUM = 5;
    reg clk;
    reg [7:0] num1, num2, correct;
    wire [7:0] out;

    MAX2 max2 (
        .num1(num1),
        .num2(num2),
        .max(out)
    )
endmodule

```

```

);

initial begin
    clk = 0;
    forever begin
        #10;
        clk = ~clk;
    end
end

integer index, fid;
initial begin
    index = 0;
    fid = $fopen("test_data.txt", "r");
    $display("[Testbench]: ===== Ready to start the test.
=====");
    repeat(TEST_NUM) begin
        @(posedge clk);
        $fscanf(fid, "%d, %d, %d", num1, num2, correct);
        index = index + 1;
    end
    #50;
    $fclose(fid);
    $display("[Testbench]: ===== Done. =====");
    $finish;
end

always @(posedge clk) begin
    #1;
    if (out != correct) begin
        $display("[Module MAX2]: Time %0t: Found ERROR at testcase
No.%0d", $time, index);
    end
end
endmodule

```

测试过程：使用 [Vivado](#) 进行仿真 - Digital Lab 2024 ([ustc.edu.cn](#))

Chapter IV 上板运行

消抖：

```
module Jitter_Clear(  
    input          clk,  
    input          btn,  
    output         btn_clean  
);  
reg [3:0] cnt;  
  
always @(posedge clk) begin  
    if (!btn)  
        cnt <= 4'h0;  
    else if (cnt < 4'h8)  
        cnt <= cnt + 1'b1;  
end  
  
assign btn_clean = cnt[3];  
endmodule
```

边沿检测：

同步检测与异步检测

异步检测的原理是直接将输入信号作为敏感变量加入到 `always` 基本块的头部。这种做法在大部分情况下都是极为不推荐的！在 FPGA 综合时，我们需要避免非时钟信号、复位信号等出现在敏感变量列表中。

```
module edge_capture(  
    input          clk,  
    input          rst,  
  
    input          sig_in, // signal input  
    output         pos_edge,  
    output         neg_edge  
);  
  
reg sig_r1, sig_r2;  
always @(posedge clk) begin  
    if (rst) begin
```

```

        sig_r1 <= 0;
        sig_r2 <= 0;
    end
    else begin
        sig_r1 <= sig_in;
        sig_r2 <= sig_r1;
    end
end
end
assign pos_edge = (sig_r1 && ~sig_r2) ? 1 : 0;
assign neg_edge = (~sig_r1 && sig_r2) ? 1 : 0;
endmodule

```

以上是两级寄存器比较给出结果

使用三级寄存器来避免亚稳态

计数器分频

```

module Clock_10M(
    input                clk, rst,
    output reg           led
);
    reg [3:0] cnt;
    wire pulse_10m;

    always @(posedge clk) begin
        if (rst)
            cnt <= 4'b0;
        else if (cnt >= 9)
            cnt <= 4'b0;
        else
            cnt <= cnt + 4'b1;
    end

    assign pulse_10m = (cnt == 4'h1);

    always @(posedge clk) begin
        if (rst)
            led <= 1'b0;
        else if (pulse_10m)
            led <= ~led;
    end
endmodule

```

```
end  
endmodule
```

IP分频

利用IP核的例化格式代码！

信号处理 - Digital Lab 2024 (ustc.edu.cn)

再创建模板文件

```
module Top (  
    input          clk,  
    input          rst  
);  
wire clk_10m, clk_200m, locked;  
  
myclock clock(  
    .clk_in1    (clk),  
    .clk_out1   (clk_10m),  
    .clk_out2   (clk_200m),  
    .reset      (rst),  
    .locked     (locked)  
);  
endmodule
```

```
module Top_tb();  
reg clk, rst;  
  
initial begin  
    clk = 0;  
    forever  
        #5 clk = ~clk;  
end  
  
initial begin  
    rst = 1;  
    #100 rst = 0;  
end  
  
Top top_test (  
    clk,  
    rst,  
    clk_10m,  
    clk_200m,  
    locked
```

```

        .clk(clk),
        .rst(rst)
    );
endmodule

```

有限状态机

有限状态机 - Digital Lab 2024 (ustc.edu.cn)

Moore和Mealy有限状态机

```

module FSM (
    input        clk,
    input        rst,
    // .....
    // 其他输入输出信号
);
// 状态空间位数 n
parameter WIDTH = 3;
// 状态变量
reg [WIDTH-1: 0] current_state, next_state;

// 为了便于标识，我们用局部参数定义状态的别名代替状态编码
localparam STATE_NAME_1 = 3'd0;
localparam STATE_NAME_2 = 3'd1;
// .....

// =====
// Part 1: 使用同步时序进行状态更新，即更新 current_state 的内容。
// =====
always @(posedge clk) begin
    // 首先检测复位信号
    if (rst)
        current_state <= RESET_STATE;
    // 随后再进行内容更新
    else
        current_state <= next_state;
end

// =====
// Part 2: 使用组合逻辑判断状态跳转逻辑，即根据 current_state 与
//          其他信号确定 next_state。

```

```

// =====
// 一般使用 case + if 语句描述跳转逻辑
always @(*) begin
    // 先对 next_state 进行默认赋值，防止出现遗漏
    next_state = current_state;
    case (current_state)
        STATE_NAME_1: begin
            // .....
        end
        STATE_NAME_2: begin
            // .....
        end
        default: begin
            // .....
        end
    endcase
end

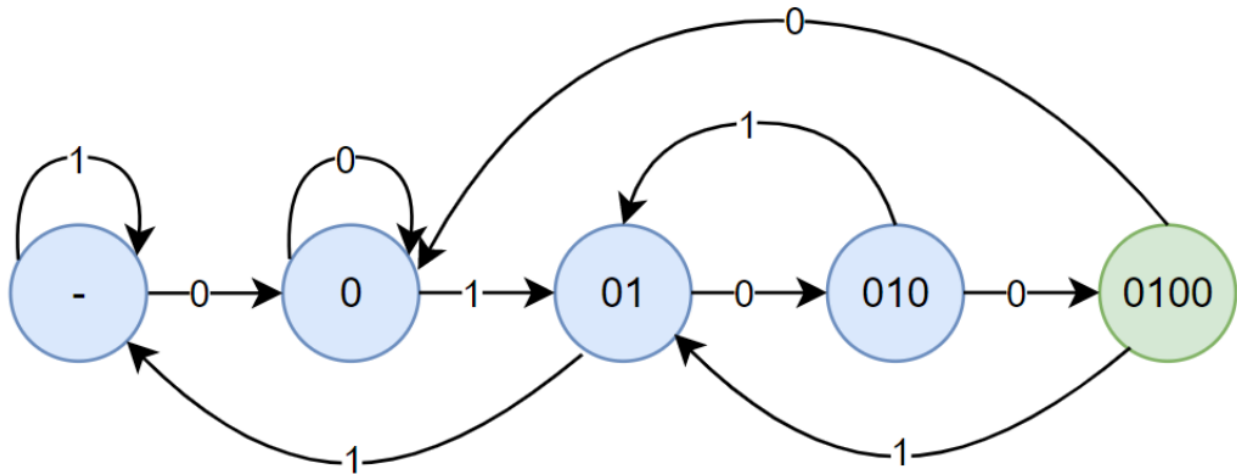
// =====
// Part 3: 使用组合逻辑描述状态机的输出。这里是 mealy 型状态机
//          与 moore 型状态机区别的地方。
// =====
// 可以直接使用 assign 进行简单逻辑的赋值
assign out1 = .....;
// 也可以用 case + if 语句进行复杂逻辑的描述
always @(*) begin
    case (current_state)
        STATE_NAME_1: begin
            // .....
        end
        STATE_NAME_2: begin
            // .....
        end
        default: begin
            // .....
        end
    endcase
end
endmodule

```

有的时候状态机内的输出信号不止一种，为了保证低耦合性，避免可能发生的错误，我们建议每一个 **always** 块内仅进行一个变量的赋值。

使用后缀识别来确定序列的正确与否，否则就有多个状态！

比如序列识别 **0100** 则作出以下的状态转移图：



再对以上的5个状态进行编码：

状态更新：

```
always @(posedge clk) begin
    if (reset)
        current_state <= S0;
    else
        current_state <= next_state;
end
```

状态转移：

```
always @(*) begin
    next_state = current_state;
    case (current_state)
        S0: begin // -
            if (in)
                next_state = S0; // -
            else
                next_state = S1; // 0
        end
    end
```

```

s1: begin // 0
    if (in)
        next_state = s2; // 01
    else
        next_state = s1; // 0
    end

s2: begin // 01
    if (in)
        next_state = s0; // -
    else
        next_state = s3; // 010
    end

s3: begin // 010
    if (in)
        next_state = s2; // 01
    else
        next_state = s4; // 0100
    end

s4: begin // 0100
    if (in)
        next_state = s2; // 01
    else
        next_state = s1; // 0
    end
endcase
end

```

可复位时钟

```

module TIMER (
    input                clk,rst,
    output [3:0]         seg_data,
    output [2:0]         seg_an
);

    reg [3:0] outm ;
    reg [3:0] outss;

```

```

reg [3:0] outsg;
reg [3:0] outst;
reg [39:0] count;

initial begin
    outm    = 4'H0;
    outss   = 4'H0;
    outsg   = 4'H0;
    outst   = 4'H0;
    count   = 40'H0;
end

```

```

always@ (posedge clk) begin
    if (rst) begin
        count <= 40'H1;
    end
    else begin
        if (count != 40'D6000000000)
            count <= count + 40'H1;
        else
            count <= 40'H1;
    end
end

```

/*使用 Lab3 实验练习中编写的数码管显示模块*/

```

Segment segment(
    .clk          (clk),
    .rst          (rst),
    .output_data  ({outm, outss, outsg, outst}),
    .output_valid (8'HFF),    // 如果你没有实现, 可以不需要这
个端口
    .seg_data     (seg_data),
    .seg_an       (seg_an)
);

```

```

always@ (posedge clk) begin
    if(rst) begin
        outm    <= 4'H0;
        outss   <= 4'H0;
        outsg   <= 4'H0;
        outst   <= 4'H0;
    end
end

```

```

        else begin
            outm    <= (count % 40'D6000000000 != 0) ? outm
            :
                (outm != 4'H5) ? outm +
1        :
                4'H0
            ;
            outss   <= (count % 40'D1000000000 != 0) ? outss
            :
                (outss != 4'H5) ? outss +
1        :
                4'H0
            ;
            outsg   <= (count % 40'D100000000 != 0) ? outsg
            :
                (outsg != 4'H9) ? outsg +
1        :
                4'H0
            ;
            outst   <= (count % 40'D10000000 != 0) ? outst
            :
                (outst != 4'H9) ? outst +
1        :
                4'H0
            ;
        end
    end
endmodule

```

移位寄存器

```

module Shift_Reg(
    input                clk,
    input                rst,
    input                en,
    input                [3:0] din,
    output                [3:0] seg_data,
    output                [2:0] seg_an,
    output                [7:0] led_sig
);

```

```

    reg [31:0] shift_reg;                                //创建 32 位寄存器分成八个四位寄
寄存器用来级联成移位寄存器
    reg [3:0] out_data;

    initial begin
        shift_reg = 32'H0;
        out_data = 4'H0;
    end

    /*使用 Lab3 实验练习中编写的数码管显示模块*/
    Segment segment(
        .clk            (clk),
        .rst            (rst),
        .output_data    (shift_reg),
        .output_valid   (8'HFF),
        .seg_data       (seg_data),
        .seg_an         (seg_an)
    );

    /*使用上面示例中展示的单个数码管显示模块*/
    Single_Segment single_segment(
        .num            (out_data),
        .led_sig        (led_sig)
    );

    wire pos_en;

    /*使用 Lab3 的边沿检测模块*/
    edge_capture en_pos_capture(
        .clk(clk),
        .rst(rst),

        .sig_in(en),
        .pos_edge(pos_en),
        .neg_edge()
    );

    always@ (posedge clk) begin
        if (en_pos) begin
            shift_reg <= {shift_reg[27:0], din};
            out_data <= shift_reg[31:28];
        end
    end

```

```
end
```

```
endmodule
```

交通信号灯

运用FSM 来设计时序逻辑电路时，不管状态机多么简单，信号量多么少，都尽量采用三段式 Verilog 代码框架进行设计

首先定义状态变量及其名称：

```
reg [1:0] current_state, next_state;  
localparam RED = 2'd0;  
localparam YELLOW = 2'd1;  
localparam GREEN = 2'd2;
```

状态更新：

```
always @(posedge clk) begin  
    if (reset)  
        current_state <= GREEN;  
    else  
        current_state <= next_state;  
end
```

状态转移：

```
always @(*) begin  
    next_state = current_state;  
    case (current_state)  
        GREEN:  
            next_state = YELLOW;  
        RED:  
            next_state = GREEN;  
        YELLOW:  
            next_state = RED;  
    endcase  
end
```

输出：

```
assign green = current_state == GREEN;  
assign yellow = current_state == YELLOW;  
assign red = current_state == RED;
```

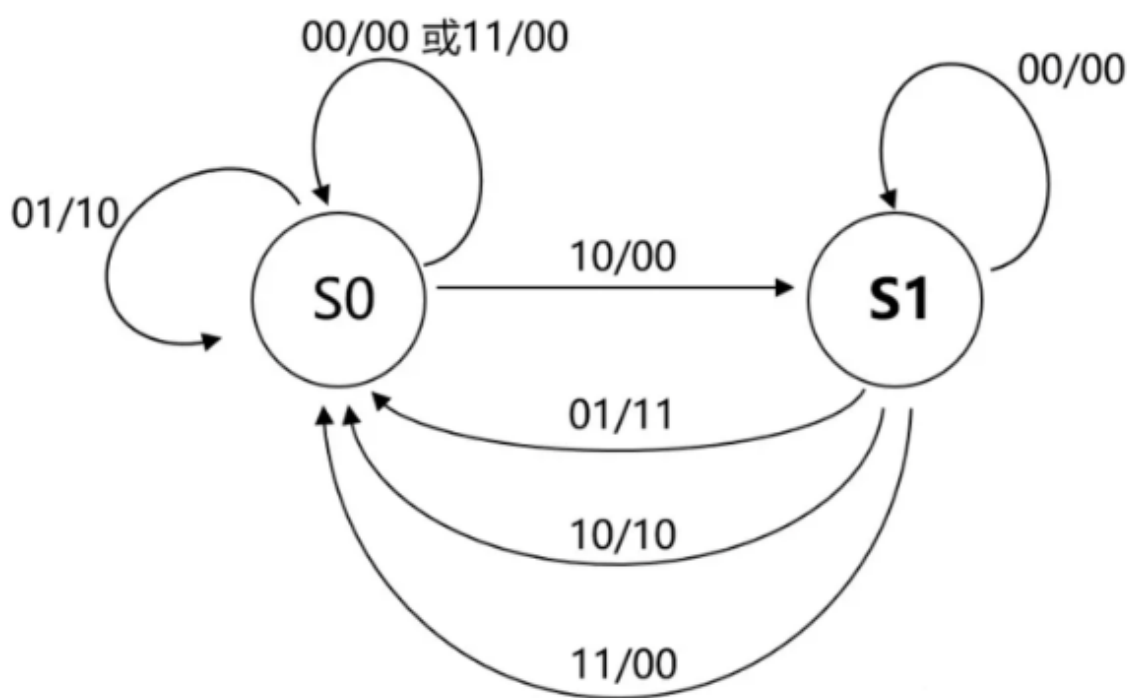
投币机

假设S0 表示售卖机里还没有钱币，S1 表示已经投了 5 分钱。

商品单价 10 分钱，硬币有 5 分和 10 分两种。假定一次只能投入一枚硬币

约定：AB/YZ

A=1 表示投入 5 分钱，B=1 表示投入 10 分钱，Y=1 表示弹出饮料，Z=1 表示找零。



画出卡诺图并进行化简

$$S^{n+1} = A \bar{B} \bar{S}^n + \bar{A} \bar{B} S^n = \bar{B} (A \bar{S}^n + \bar{A} S^n) = \bar{B} (A \oplus S^n)$$

$$Y = \bar{A} B + A \bar{B} S^n$$

$$Z = \bar{A} B S^n$$

超前进位加法器

余下内容略